

Script Presentatore — Demo "Rome Future Week" (Crm-A Console)

Percorso consolidato della demo, beat per beat, derivato da `DEMO-RUNBOOK.md`.
Per ogni passaggio: **cosa fa il presentatore**, **cosa dice**, e **cosa vede il pubblico**. Durata indicativa: ~25–30 min. Tono: vendita/educazione, ritmo tre atti "problema → memoria → orchestrazione".

0. Apertura & setup (5 min) — "Cosa vedrete"

 **Scena:** schermo condiviso, terminale + console aperta su `localhost:3100`. Tutto già avviato e seedato. Il segreto webhook e il dial outbound sono già valorizzati in `.env`.

Azione presentatore:

```
# già eseguito prima della demo — NON ridigitare dal vivo se non necessario
export CRM_A_PHONE_WEBHOOK_SECRET=<secret-demo>
curl -X POST localhost:3100/api/demo/seed -H "Authorization: Bearer
$CRM_A_PHONE_WEBHOOK_SECRET"
```

Mostra la console: **Integrations** → **card NLPearl** (credenziali + webhook URLs) e **Skills** → **nlpearl**.

Script:

"Buongiorno. Quello che vedete è Crm-A Console: un CRM-AI interamente locale, che non è solo un database di contatti: è un *cervello* che parla con i vostri sistemi di comunicazione — telefono, email, Telegram. Oggi vi mostro il filo completo di una vera campagna di lancio: un lead che arriva dalla prima chiamata, diventa memoria nel CRM, viene orchestrato su più canali e — novità di oggi — lanciamo una campagna telefonica *parlando con l'agente in chat*, senza scrivere una riga di API."

(Mostra card NLPearL e skill nlpearl.) "Questo è il punto di ingresso: le credenziali NLPearl — il motore che fa le telefonate con l'AI — e la skill che insegna all'agente come usarlo."

👁️ **Pubblico vede:** card NLPearl (Account ID/Secret/webhook); skill "nlpearl" nella Skill Store.

Atto 1 — Primo contatto: il CRM crea la persona (5 min)

Obiettivo: mostrare che la Console, da un solo numero sconosciuto, in automatico crea il profilo e gestisce il consenso.

💻 **Azione:** incollare i due `curl` (1a in ingresso, 1b a fine chiamata) e mostrare il profilo "Lorenzo" in console.

```
curl -sX POST localhost:3100/api/webhooks/phone -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \
  -d '{"action":"inbound","callId":"vc-1","from":{"phone":"+39311222333"}}'
curl -sX POST localhost:3100/api/webhooks/phone -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \
  -d '{"action":"completed","callId":"vc-1","from":{"phone":"+39311222333"},"durationSec":150,
      "data":
      {"name":"Lorenzo","email":"lorenzo@example.com","preferredContact":"telegram",
       "marketingOptIn":true,"summary":"Vuole essere avvisato al lancio Galaxy"}}'
```

🎤 Script:

"Primo scenario: un numero sconosciuto chiama. La parte telefonica fa la conversazione con l'AI, ma *qui* — nella Console — avviene la magia: il numero non esiste, quindi il CRM lo crea al volo e restituisce il contesto all'AI. Alla fine della chiamata registriamo l'esito: Lorenzo ha dato il consenso, preferisce Telegram, vuole essere avvisato al lancio del Galaxy. Un secondo, ed è già un contatto pieno, con la sua interazione in timeline."

👁️ **Pubblico vede:** risposta JSON { `matched:"created"` }; profilo Lorenzo con Preferenza=telegram, Opt-in=true; interazione `Call` nella timeline.

Atto 2 — Il CRM diventa memoria (Copilot) (3 min)

Obiettivo: mostrare che l'agente ragiona sui dati, non serve una query.

 **Azione:** aprire la chat (Copilot) e digitare la domanda.

"Stiamo lanciando il nuovo Samsung Galaxy. Qual è il pubblico migliore a cui proporlo?"

 **Script:**

"Ora il CRM ragiona. Non c'è una query scritta da un DBA: l'agente usa la sua skill CRM, legge il segmento di lancio, incrocia chi ha già acquistato i modelli precedenti e le preferenze di canale, e risponde. Guardate: Lorenzo è nella lista, perché ha opt-in e canale preferito."

(L'agente risponde elencando il pubblico.) "Questo è il potere di avere una base dati viva, non un foglio Excel."

 **Pubblico vede:** risposta dell'agente con la lista del pubblico; ragionamento sulle preferenze di canale.

Atto 3 — Orchestrazione multicanale (5 min)

 **Azione:** esportare il brief + invio multicanale, mostrando la segmentazione per canale.

```
curl -s -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" \  
  "localhost:3100/api/marketing/brief?promotedSku=SAM-S27&previousSku=SAM-S26" -o brief-galaxy-s27.md  
curl -sX POST localhost:3100/api/campaigns/send-multichannel -H \  
  "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \  
  -d '{"segmentEntryId":"<id segmento>","subject":"Samsung Galaxy 27","body":"Disponibile dal 18 ottobre. Promo per i primi clienti."}'
```

 **Script:**

"Ora orchestriamo. Il prodotto 'viaggia' come un brief di marketing: caratteristiche, differenze col modello precedente, promo. E spediamo la campagna **per canale preferito** — chi ha scelto Telegram lo riceve su Telegram, chi email su email. Uno stesso messaggio, instradato automaticamente. Vedi il risultato: `sent`, i destinatari per canale."

👁️ **Pubblico vede:** file `brief-galaxy-s27.md`; JSON `{ sent, telegram:[...], email:[...] }` — chi a chi.

Atto 4 — Il cliente conversa (3 min)

💻 **Azione:** messaggio in ingresso di Lorenzo che chiede se "vale la pena".

```
curl -sX POST localhost:3100/api/webhooks/phone -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \
  -d '{"action":"message","messageId":"tg-1","contact":
{"phone":"+393312345678","name":"Lorenzo"},
"text":"Grazie, l''offerta mi interessa. Vale la pena?"}'
```

🎙️ **Script:**

"Lorenzo risponde. La Console gli restituisce identità e storico; il *contenuto* del prodotto — perché il Galaxy 27 vale la pena — viene dal brief che l'ambiente telefonico ha importato all'atto tre. Questa è l'architettura: chi parla usa la conoscenza, chi ricorda è il CRM."

👁️ **Pubblico vede:** JSON `{ person, context }` — l'AI risponde citando il brief.

Atto 5 — Ricordarsi del cliente dopo l'acquisto (3 min)

💻 **Azione:** inbound di Lorenzo a numero noto.

```
curl -sX POST localhost:3100/api/webhooks/phone -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \
```

```
-d '{"action":"inbound","callId":"vc-5","from":  
{"phone":"+393312345678","name":"Lorenzo"}}'
```

Script:

"E adesso il tocco finale: Lorenzo richiama. La Console sa chi è, e soprattutto sa *l'ultimo acquisto*: il corriere è in carico, consegna prevista domani entro le 18. L'AI lo saluta per nome e gli dà lo stato dell'ordine. Niente più 'chi parla?'. È questo il CRM che *ricorda*."

👁️ **Pubblico vede:** JSON con `lastOrder` (prodotto, corriere GLS, deliveryStatus); la frase di benvenuto personalizzata.

★ Atto 6 — NOVITÀ: campagna outbound creata chattando (6 min) (*il momento clou*)

Obiettivo: mostrare il tool `crm_a_phone_campaign` — l'agente che, da solo, crea la scheda e pilota NLPearl, con l'operatore che dà solo le conferme.

 **Azione:** in chat, digitare:

"Crea una campagna outbound per il Galaxy 27, con una breve comparazione col modello precedente, e chiama chi preferisce il telefono."

Script (beat):

(L'agente chiama il tool con `upsert`.) "Guardate cosa succede in automatico: l'agente crea la **scheda campagna** — prodotto, confronto con il modello precedente, config telefonica. E non vi mostra codice: lo fa lui, dietro le quinte."

(L'agente chiama `create`.) "Ora crea la **Pearl su NLPearl** — l'agente vocale. Nota: parte **in pausa**, nessuna chiamata finché non confermo io. Questo è il punto: il sistema non scatta da solo."

(L'agente mostra l'anteprima audience e chiede conferma per `send` — mai in automatico.) "Ora definisce *chi chiamare*: solo chi ha dato il consenso telefonico. E **chiede a me la conferma prima di mandare i lead**. Nessun numero viene chiamato senza il via dell'operatore."

(Confermare, poi mostrare l'output `leadsCreated`.) "Ecco i lead enqueued. La campagna è pronta — ancora in pausa."

(Opzionale, solo con numero di test tuo: confermare `resume`.) "Quando l'operatore dà l'ok esplicito — e solo allora — la Pearl inizia a chiamare. Ogni esito torna al CRM e aggiorna il record: chi ha risposto, chi ha comprato, chi va richiamato."

👁️ **Pubblico vede:** i tool-call dell'agente in chat (`upsert` → `create` → `send`); la scheda campagna con Voice Brief; il gate di conferma `confirm: true`; `leadsCreated`.

Regole da rispettare sul palco:

- `send/resume` **solo** dietro conferma esplicita (`confirm: true`) — mai numeri seed/demo.
- Per un invio reale serve il tuo numero di test + origin pubblico (`CRM_A_CONSOLE_PUBLIC_URL/tunnel`) per i callback.
- Tienilo "paused" per la maggior parte della demo; la parte `resume` va solo se collaudata prima.

Chiusura (2 min) — Call to action

🗣️ Script:

"Ricapitolando in venti minuti: un lead nasce da una chiamata, diventa un contatto vivo, l'agente lo segmenta come pubblico, una campagna multicanale lo raggiunge sul canale che preferisce, il CRM lo riconosce a ogni richiamo — e oggi, la novità, l'operatore lancia una campagna telefonica **parlando in chat**, con il controllo umano sempre al centro. Crm-A Console: il CRM che ascolta, ricorda e agisce. Domande?"

💻 **Azione:** aprire le domande; tenere pronto il `DEMO-RUNBOOK.md` per i dettagli tecnici e le "Note NLPearl" (phone id, variabili, prerequisito origin) se qualcuno chiede come è costruito.

Checklist pre-demo (una scorsa prima di salire)

- ☐ `pnpm build:crm-a-plugins + docker compose up -d --build --force-recreate` (immagine con card NLPearL + skill nlpearl + estensione)
- ☐ `.env` con `CRM_A_PHONE_WEBHOOK_SECRET` (+ `NLPEARL_ACCOUNT_ID/SECRET_KEY`) — caricato via `env_file`
- ☐ `/api/integrations` risponde 200; card NLPearL visibile; skill "nlpearl" in Installed
- ☐ Seed eseguito (`/api/demo/seed`)
- ☐ Origin pubblico attivo (`CRM_A_CONSOLE_PUBLIC_URL` o tunnel) per i callback della parte live
- ☐ Numero di test tuo se prevedi `send/resume` reali
- ☐ Hard refresh browser, terminale pulito e pronto con i `curl`